

1 Algoritmo di enumerazione dei rettangoli di una facciata

Data una facciata di un edificio contenente porte e/o finestre, il prolungamento dei lati di queste fino ai bordi della facciata genera una griglia di rettangoli, che chiamiamo celle. Si vogliono contare e memorizzare tutti i rettangoli ammissibili presenti, dove per rettangolo si intende l'unione di una o più celle adiacenti - a patto che la figura finale sia ancora un rettangolo - e ammissibili significa non contenenti (porzioni di) porte o finestre.

Si rappresenta la griglia ottenuta con una matrice $A \in \mathbb{R}^{n \times m}$ di uni e zeri: uni in corrispondenza delle celle ammissibili e zeri nelle posizioni di porte e finestre. In questo modo un rettangolo ammissibile è una sottomatrice di A priva di zeri.

L'obiettivo dell'algoritmo è enumerare tutte le sottomatrici prive di zeri. A tale scopo ciascuna sottomatrice viene memorizzata come una quadrupla (i, j, i_2, j_2) dove (i, j) indica la posizione del primo elemento della sottomatrice (quello in alto a sinistra) e (i_2, j_2) quella dell'ultimo elemento in basso a destra. Analogamente, il rettangolo (i, j, i_2, j_2) è il rettangolo con vertice superiore sinistro in (i, j) e vertice inferiore destro in (i_2, j_2) . Si veda la Figura 2 per alcuni esempi.

1.1 Spiegazione dell'algoritmo

L'algoritmo utilizza quattro cicli annidati per esplorare tutte le possibili coppie di coordinate che definiscono una sottomatrice priva di zeri (equivalentemente, un rettangolo ammissibile).

In particolare:

- I primi due cicli, indicizzati da i e j , selezionano la cella in alto a sinistra del rettangolo, ovvero la cella di partenza.
- Per ogni coppia (i, j) , viene inizializzato un vettore booleano **valide** di lunghezza m , inizialmente contenente tutti valori **True**. Questo vettore serve a memorizzare quali colonne possono ancora far parte di un rettangolo ammissibile, così da evitare iterazioni superflue.
- Il terzo ciclo, indicizzato da i_2 , estende progressivamente il rettangolo verso il basso ($i_2 \geq i$). Ad ogni iterazione, il vettore **valide** viene aggiornato, sostituendo il j -esimo elemento con **False**, se in posizione (i_2, j) è presente uno 0. In questo modo, il vettore memorizza quali colonne sono valide, ossia prive di zeri fino a quel punto, al fine di interrompere l'estensione del rettangolo verso destra (quarto ciclo) non appena si incontra una colonna j_2 che contiene almeno uno zero nelle posizioni tra (i, j_2) e (i_2, j_2) .
- Il quarto ciclo, indicizzato da j_2 , estende il rettangolo verso destra. Per ogni colonna $j_2 \geq j$:

- Se `valide[j_2]` è `True`, allora il rettangolo definito dalle coordinate (i, j) e (i_2, j_2) è ammissibile e viene aggiunto alla lista dei risultati.
- Se invece `valide[j_2]` è `False`, significa che esiste almeno uno zero in quella colonna nell'intervallo di righe considerato. In tal caso, il ciclo viene interrotto tramite `break`, poiché tutte le colonne successive non possono formare rettangoli ammissibili.

Nella seguente sezione si riporta un esempio di applicazione dell'algoritmo.

2 Esempio

Si consideri la facciata di un edificio riportata in Figura 1, contenente due finestre e una porta.

	0	1	2	3	4	5
0	1	1	1	1	1	1
1	1	0	1	1	1	0
2	1	1	1	1	1	1
3	1	1	1	0	1	1

Figura 1: Facciata dell'edificio e griglia di uni e zeri. Esternamente sono riportati gli indici (i, j) , corrispondenti alle posizioni nella matrice A .

La matrice $A \in \mathbb{R}^{4 \times 6}$ corrispondente è la seguente:

$$\begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 & 1 \end{pmatrix}$$

Si riportano esplicitamente i primi passi dell'algoritmo.

Passo 1: scelta della cella iniziale (i, j)

- Partiamo da: $(i, j) = (0, 0)$.
- Inizializziamo il vettore: `valide` = $(True, True, True, True, True, True)$.

Passo 2: scelta della riga finale $i_2 \geq i$

- Selezioniamo $i_2 = 0$ come riga finale.
- $A[0] = (1, 1, 1, 1, 1, 1)$. Non ci sono zeri quindi $\mathbf{valide} = (True, True, True, True, True, True)$.

Passo 3: estensione verso destra $j_2 \geq j$

- Variamo l'indice di colonna finale j_2 fintanto che $\mathbf{valide}[j_2] = True$, ottenendo le seguenti quadruple: $(0, 0, 0, 0)$, $(0, 0, 0, 1)$, $(0, 0, 0, 2)$, $(0, 0, 0, 3)$, $(0, 0, 0, 4)$, $(0, 0, 0, 5)$, che rappresentano rispettivamente le sottomatrici (1) , $(1, 1)$, $\dots$, $(1, 1, 1, 1, 1, 1)$ nella prima riga in alto di A , partendo dalla cella $(0, 0)$ in alto a sinistra.

Passo 4: aggiornamento riga finale

- Terminato il ciclo su j_2 , si aggiorna la riga finale così da estendere verso il basso i rettangoli cercati: $i_2 = 1$.
- $A[i_2] = (1, 0, 1, 1, 1, 0)$. Poichè la riga i_2 -esima contiene due zeri, l'aggiornamento produce $\mathbf{valide} = (True, False, True, True, True, False)$

Passo 5: estensione verso destra

- $j_2 = 0$: $\mathbf{valide}[0] = True \Rightarrow$ quadrupla $(0, 0, 1, 0)$ ovvero sottomatrice $\begin{pmatrix} 1 \\ 1 \end{pmatrix}$ a partire dall'angolo in alto a sinistra.
- $j_2 = 1$: $\mathbf{valide}[1] = False \Rightarrow$ il ciclo su j_2 si ferma e si torna all'aggiornamento della riga finale i_2 . In altre parole, la sottomatrice $\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$ non è ammissibile e non lo saranno nemmeno tutte le "successive", ossia quelle ottenute estendendo le colonne verso destra.

Passi successivi

- Una volta terminato l'aggiornamento delle righe finali, l'algoritmo torna a definire una nuova cella iniziale (i, j) , variando dapprima l'indice di colonna j .
- Si riparte da $(i, j) = (0, 1)$
- Si inizializza nuovamente il vettore $\mathbf{valide} = (True, True, True, True, True, True)$
- Si ripete dal Passo 2

3 Risultati dell'algoritmo

L'algoritmo applicato all'istanza di Figura 1 individua 94 rettangoli ammissibili, ciascuno rappresentato da una quadrupla (i, j, i_2, j_2) . Si riportano tutti i rettangoli nell'Appendice 3.

A titolo esemplificativo si riportano in Figura 2 il rettangolo $(0, 0, 0, 0)$ in verde, il rettangolo $(2, 3, 2, 5)$ in rosso e il rettangolo $(0, 2, 1, 3)$ in blu. Come già detto, osservando la figura, la quadrupla è da interpretarsi così: i primi due indici indicano che il rettangolo individuato ha il vertice in alto a sinistra coincidente col vertice in alto a sinistra della cella (i, j) . Analogamente per il vertice in basso a destra, indicato dagli ultimi due indici (i_2, j_2) . Ciò determina univocamente ciascun rettangolo.

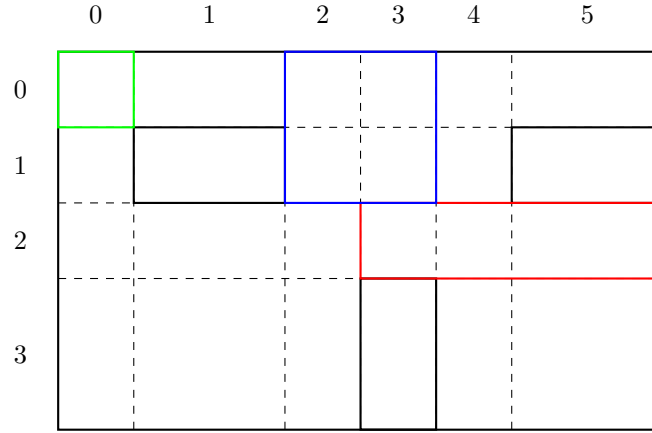


Figura 2: Alcuni rettangoli individuati dall'algoritmo.

A Output dell'algoritmo

```
(0, 0, 0, 0)
(0, 0, 0, 1)
(0, 0, 0, 2)
(0, 0, 0, 3)
(0, 0, 0, 4)
(0, 0, 0, 5)
(0, 0, 1, 0)
(0, 0, 2, 0)
(0, 0, 3, 0)
(0, 1, 0, 1)
(0, 1, 0, 2)
(0, 1, 0, 3)
(0, 1, 0, 4)
(0, 1, 0, 5)
(0, 2, 0, 2)
(0, 2, 0, 3)
(0, 2, 0, 4)
(0, 2, 0, 5)
(0, 2, 1, 2)
(0, 2, 1, 3)
(0, 2, 1, 4)
(0, 2, 2, 2)
(0, 2, 2, 3)
(0, 2, 2, 4)
(0, 2, 3, 2)
(0, 3, 0, 3)
(0, 3, 0, 4)
(0, 3, 0, 5)
(0, 3, 1, 3)
(0, 3, 1, 4)
(0, 3, 2, 3)
(0, 3, 2, 4)
(0, 4, 0, 4)
(0, 4, 0, 5)
(0, 4, 1, 4)
(0, 4, 2, 4)
(0, 4, 3, 4)
(0, 5, 0, 5)
(1, 0, 1, 0)
(1, 0, 2, 0)
(1, 0, 3, 0)
(1, 2, 1, 2)
(1, 2, 1, 3)
(1, 2, 1, 4)
(1, 2, 2, 2)
(1, 2, 2, 3)
(1, 2, 2, 4)
```

(1, 2, 3, 2)
(1, 3, 1, 3)
(1, 3, 1, 4)
(1, 3, 2, 3)
(1, 3, 2, 4)
(1, 4, 1, 4)
(1, 4, 2, 4)
(1, 4, 3, 4)
(2, 0, 2, 0)
(2, 0, 2, 1)
(2, 0, 2, 2)
(2, 0, 2, 3)
(2, 0, 2, 4)
(2, 0, 2, 5)
(2, 0, 3, 0)
(2, 0, 3, 1)
(2, 0, 3, 2)
(2, 1, 2, 1)
(2, 1, 2, 2)
(2, 1, 2, 3)
(2, 1, 2, 4)
(2, 1, 2, 5)
(2, 1, 3, 1)
(2, 1, 3, 2)
(2, 2, 2, 2)
(2, 2, 2, 3)
(2, 2, 2, 4)
(2, 2, 2, 5)
(2, 2, 3, 2)
(2, 3, 2, 3)
(2, 3, 2, 4)
(2, 3, 2, 5)
(2, 4, 2, 4)
(2, 4, 2, 5)
(2, 4, 3, 4)
(2, 4, 3, 5)
(2, 5, 2, 5)
(2, 5, 3, 5)
(3, 0, 3, 0)
(3, 0, 3, 1)
(3, 0, 3, 2)
(3, 1, 3, 1)
(3, 1, 3, 2)
(3, 2, 3, 2)
(3, 4, 3, 4)
(3, 4, 3, 5)
(3, 5, 3, 5)